



11-ERROR-TAXONOMY.md — One error model for Rust, TypeScript and Python



Rule R1 — No silent failure. Every failure in AURA has a stable code, an owner, a severity, a user-facing sentence and a defined recovery action. If a failure cannot be described in those five terms, it is not allowed to exist.

1. Mission

Give the entire codebase one error vocabulary so that a failure at file 2,847 of a 3,000-image autopilot run is *classified, recovered from, logged and explained* rather than crashing the wedding.

2. Error code format

`AURA-<DOMAIN>-<NNNN>` — immutable once shipped. Codes are never reused, never renumbered. Retired codes are marked `DEPRECATED` in the registry and kept forever.

Domain	Range	Owner	Example
<code>IO</code>	1000–1999	SRC	<code>AURA-IO-1004</code> card disconnected mid-import
<code>RAW</code>	2000–2999	TLC	<code>AURA-RAW-2011</code> unsupported CR3 variant
<code>DB</code>	3000–3999	SRC	<code>AURA-DB-3002</code> catalog schema newer than app

Domain	Range	Owner	Example
GPU	4000–4999	SRG	AURA-GPU-4001 device lost during render
ML	5000–5999	MLL	AURA-ML-5007 ONNX output shape mismatch
CLOUD	6000–6999	AGT	AURA-CLOUD-6003 budget cap reached
JOB	7000–7999	TLC	AURA-JOB-7005 task exceeded wall-clock budget
EXPORT	8000–8999	MBE	AURA-EXPORT-8002 destination out of disk space
SEC	9000–9999	SEC	AURA-SEC-9001 keychain unavailable

3. Severity and autonomy

Severity	Meaning	Autopilot behaviour	UI
Fatal	Catalog integrity or user data at risk	Stop run, checkpoint, refuse to continue	Blocking dialog + run bundle
RunBlocking	The wedding cannot finish	Pause run, keep completed work	Banner with retry
ItemFailed	One photo failed, gallery is fine	Quarantine the item, continue	Review queue chip
Degraded	Fell back to a lesser path	Continue, record in decision ledger	Info badge on affected photos
Warning	Worth telling a human later	Continue silently	Run report only

Hard rule: an `ItemFailed` may never escalate itself into a stopped run. A run stops only when `Fatal` / `RunBlocking` fires, or when the *item failure rate* exceeds 2 % of processed files — which raises `AURA-JOB-7009 SystemicItemFailure` at `RunBlocking`.

4. Frozen Rust contract — copy verbatim

```

// crates/aura-core/src/error.rs — FROZEN CONTRACT v1
#[derive(Debug, Clone, Copy, PartialEq, Eq, serde::Serialize,
serde::Deserialize)]
pub enum Severity { Fatal, RunBlocking, ItemFailed, Degraded,
Warning }

#[derive(Debug, Clone, Copy, PartialEq, Eq, serde::Serialize,
serde::Deserialize)]
pub enum Recovery {
    /// Retry with exponential backoff, jitter, max attempts.
    Retry { max_attempts: u8, base_ms: u32 },
    /// Use the named fallback path (e.g. "cpu_ep", "embedded
_preview", "local_only").
    Fallback(&'static str),
    /// Skip this item, quarantine it, continue the run.
    Quarantine,
    /// Stop the run at a clean checkpoint.
    Halt,
    /// Ask the photographer.
    AskUser,
}

#[derive(Debug, thiserror::Error)]
#[error("{code}: {detail}")]
pub struct AuraError {
    pub code: &'static str,           // "AURA-RAW-2011"
    pub severity: Severity,
    pub recovery: Recovery,
    pub detail: String,               // engineer-facing, may
contain paths
    pub user_message: &'static str, // photographer-facing,
no jargon, no paths
    pub context: ErrorContext,
    #[source]
    pub source: Option<Box<dyn std::error::Error + Send + Syn

```

```

c>>,
}

#[derive(Debug, Clone, Default, serde::Serialize, serde::Deserialize)]
pub struct ErrorContext {
    pub project_id: Option<i64>,
    pub photo_id: Option<i64>,
    pub job_id: Option<String>,
    pub phase: Option<&'static str>, // "cull", "develop",
    "retouch", "export"
    pub model_version: Option<String>,
    pub execution_provider: Option<String>,
    pub attempt: u8,
}

pub type AuraResult<T> = Result<T, AuraError>;

```

5. The registry file

Every code lives in `crates/aura-core/errors.toml`, which is the single source of truth. CI generates the Rust constants, the TypeScript union type and the docs table from it — they are never hand-written in three places.

```

[[error]]
code = "AURA-GPU-4001"
name = "DeviceLost"
severity = "Degraded"
recovery = { type = "Fallback", target = "cpu_develop" }
user_message = "Your graphics card restarted. AURA switched to a slower but identical CPU path and is continuing."
owner = "SRG"
introduced = "0.3.0"
runbook = "docs/runbooks/gpu-device-lost.md"

```

6. Banned patterns — CI fails the build

Pattern	Why it is banned	Do this instead
<code>.unwrap()</code> / <code>.expect()</code> outside <code>#[cfg(test)]</code> and <code>main()</code>	Turns a recoverable item failure into a crashed wedding	<code>?</code> with a mapped <code>AuraError</code>
<code>panic!()</code> , <code>todo!()</code> , <code>unimplemented!()</code> in shipped paths	Same	Return <code>AURA-JOB-7001 NotImplemented</code> at <code>ItemFailed</code>
<code>let _ = fallible();</code>	Silent swallow	Log at minimum <code>Warning</code> with a code
<code>anyhow::Error</code> crossing a crate boundary	Loses the code and severity	Convert to <code>AuraError</code> at the boundary
String-matched errors in the UI	Breaks on i18n and rewording	Switch on <code>code</code>
<code>catch {}</code> with empty body in TS	Silent swallow	<code>reportError(code, ctx)</code>
Bare <code>except:</code> in Python training code	Hides CUDA OOM as "worked"	<code>except SpecificError</code> • re-raise

Enforcement: `clippy.toml` with `disallowed-methods`, plus a `scripts/check-banned.sh` grep gate wired into `20-CI-PIPELINE-SPEC.md`.

7. Retry policy (frozen)

Class	Retry?	Attempts	Backoff
Transient IO (network drive, card hiccup)	Yes	3	250 ms × 2 ⁿ ± 20 % jitter
GPU device lost	Once, then fall back to CPU	1	Immediate
Cloud 429 / 5xx	Yes, honour <code>Retry-After</code>	4	1 s × 2 ⁿ , cap 30 s
Cloud 4xx (bad request, auth)	Never	0	—

Class	Retry?	Attempts	Backoff
Corrupt RAW	Never	0	Quarantine immediately
SQLite <code>BUSY</code>	Yes	10	5 ms × n, then <code>Halt</code>

Retries must be idempotent. Any operation registered as retryable must pass the idempotency test in `15-CONCURRENCY-AND-JOB-GRAPH.md`.

8. Quarantine

A quarantined photo is not a lost photo:

- Row written to `photo_quarantine (photo_id, code, detail, attempt, first_seen, last_seen)`.
- The file appears in a **Needs attention** queue with the plain-English `user_message` and a one-click *Retry / Import as-is / Exclude from gallery*.
- The autopilot final report states: `3,000 imported · 2,994 delivered · 6 need attention` — never a silent 2,994.
- Delivery is blocked if quarantined items exceed the configured tolerance (default 1 %).

9. Test plan

- **Unit** — every `Recovery` variant has a test proving the recovery actually happens.
- **Registry** — test asserts every code in `errors.toml` is reachable from at least one code path (dead-code detection for errors).
- **Fault injection** — `aura-faults` crate injects each domain's top failures via env var; used by the chaos suite in `14-FMEA-FAILURE-CATALOG.md`.
- **Golden** — snapshot test of the run report for the "disaster wedding" fixture: corrupt files, mixed cameras, disconnected drive.
- **i18n** — test asserts every `user_message` has a translation key and contains no file paths, no stack traces, no error codes.

10. Acceptance criteria

- ☐ `errors.toml` exists; Rust and TS bindings are generated, not hand-written.
- ☐ Zero `unwrap` / `expect` / `panic` outside tests — CI-enforced.
- ☐ Every `AuraError` construction site supplies a real `ErrorContext`.
- ☐ The disaster-wedding fixture completes end-to-end with zero crashes and a correct quarantine report.
- ☐ Killing the app during any injected fault leaves a valid, resumable catalog.

11. Claude Code prompt

Act as `TLC` (Tech Lead — Imaging Core). Implement `crates/aura-core/src/error.rs` exactly as the frozen contract in `11-ERROR-TAXONOMY.md`. Then create `errors.toml` with the initial code set for domains IO, RAW and DB, and write `build.rs` codegen producing `error_codes.rs` and `errorCodes.ts`. Do not invent codes outside the assigned ranges. Then switch hats to `QAL` and write the registry reachability test. Report PASS/FAIL with command output.